

Reading Exercise 03 — Following a Token

Pick up one C keyword. Follow it from the source file through the preprocessor into the parser and out the other side. Stop when you lose the thread.

What this exercise trains

A compiler's job is to transform. The same input appears in different forms as it moves through a compiler's stages. Learning to follow one piece of input through those transformations is one of the most useful reading skills you can develop. It gives you a concrete path through code that would otherwise be a maze.

The keyword is `int`. Simple, common, unavoidable. Every C program you have ever written contains it.

Step 1 — Where tokens are defined [5 minutes]

Open `practice/tcc/tcc.h`. Search for the token definitions:

```
grep -n "TOK_INT\\|\"int\"\\|kw_int" practice/tcc/tcc.h | head -20
```

Find where the keyword `int` is assigned a token number. It may appear as an entry in a keyword table or as a `#define`.

Write down:

- The line number where `int` as a keyword is defined or registered
 - The token value or name it gets
 - How many keywords does TCC recognize in total? (Count the entries near the `int` definition.)
-

Step 2 — Where keywords are recognized [10 minutes]

The preprocessor reads source text and produces a token stream. Open `practice/tcc/tccpp.c`. This file handles tokenization.

Search for where keyword recognition happens:

```
grep -n "keyword\|TOK_INT\|is_ident\|hash" practice/tcc/tccpp.c | head -20
```

Find the function that converts a word from the source file into a token. Read it for 5 minutes.

Write down:

- The function name and line number
 - How does TCC decide that the characters `i`, `n`, `t` in sequence represent the keyword `int` (and not just a variable named `int`)?
 - What does the function return or produce?
-

Step 3 — Where the token is consumed [10 minutes]

The parser in `practice/tcc/tccgen.c` receives the token stream and does something with `TOK_INT`. Find the consumer:

```
grep -n "TOK_INT" practice/tcc/tccgen.c | head -20
```

Find the case or branch that handles `TOK_INT` as a type specifier. Read what happens when the parser sees `int`.

Write down:

- The line number
 - What data structure does TCC build when it sees `int`? (It represents a C type internally — what does that representation look like?)
 - Find the word "CType" or "type" in this context. What fields does a type have in TCC?
-

Step 4 — Where the type becomes code [10 minutes]

When TCC compiles a declaration like `int x;`, it eventually allocates storage for `x`. Find where this happens — where a type representation becomes a concrete allocation of bytes or a register.

This is harder to trace. Start from the `TOK_INT` case and follow the next significant function call:

```
grep -n "sym_push\|put_extern_sym\|gv\|vstore" practice/tcc/tccgen.c | head -30
```

Read for 5 minutes.

Write down:

- What function handles variable allocation or declaration?
 - How does TCC decide how many bytes `int` occupies? Where is that size stored or computed?
 - At what point does the word `"int"` disappear and become a number?
-

Step 5 — The full path [5 minutes]

Draw the path of the keyword `int` through TCC's pipeline:

```
Source text "int"
→ [tokenizer, tccpp.c] → TOK_INT
→ [parser, tccgen.c]   → type representation
→ [code gen, tccgen.c] → storage allocation
→ [emitter]            → machine code
```

Fill in function names at each stage based on what you found.

Write down: where in the pipeline did you lose the thread? That is exactly the right place to read next.

Debrief

You have followed one word through four stages of compilation. You did not understand everything — probably not even most of it. That is correct.

What you have is a thread. A specific, anchored path through the code. The next time you open `tccgen.c` you will recognize the neighborhood around `TOK_INT` because you have been there. The compiler will be less foreign.

That is the goal of this exercise. Not comprehension. Familiarity. Familiarity comes first; comprehension builds on it.

Next: RE-04 — The Comment Hunt — what comments reveal about what the code cannot say.